

LAB 1:

Date: 17 de mayo de 2025



Professor:

Dr. Francisco Javier Rodríguez Navarrete

Students:

Alonso Emmanuel López Macias

César Eduardo Inda Cenicerós.

Team: 14

Introduction

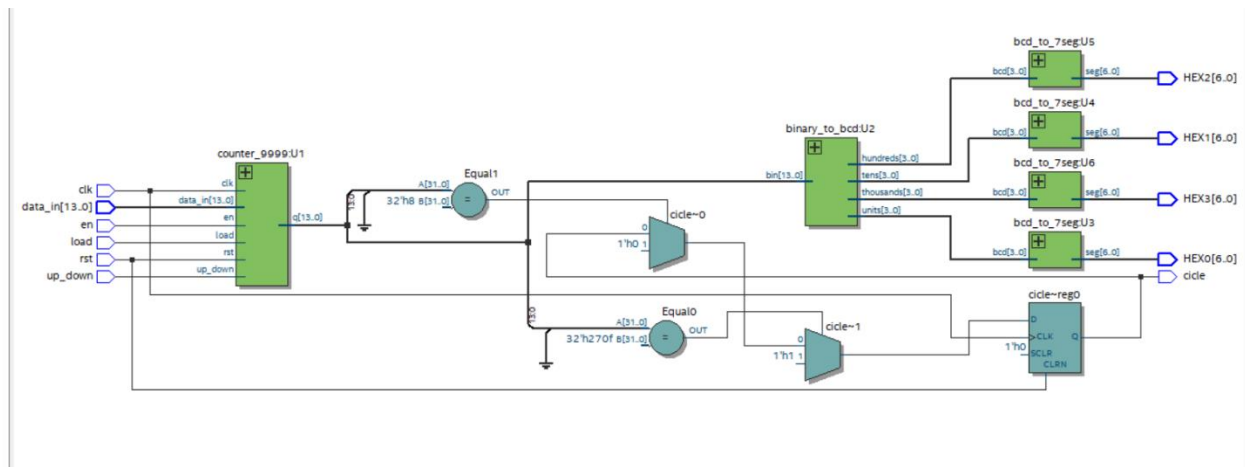
In this lab, a sequential digital counter is developed in Verilog, capable of counting from 0 to 9999, both upwards and downwards. This type of circuit is fundamental in digital systems, as it allows controlling events, measuring time, generating sequences, and addressing memories. The project includes parameterizing the counter, integrating it with an FPGA, displaying the output on 7-segment displays, and simulating the system's behavior through a testbench.

Requirements

- Design a counter that can reach the decimal value 9999.
- The counter must count up or down based on the up_down signal.
- The initial value can be loaded via the load signal and the input data_in.
- If the loaded value exceeds 9999, the counter resets to 0.
- The output q must be displayed on 4 seven-segment displays.
- When the counter value is 9999, LEDG8 should turn on.
- When the counter value is 8, LEDG8 should turn off.
- The clk is generated by a button (KEY0) and reset is triggered by KEY1.
- The system must be verified using a testbench in simulation.

Development

Parametric Counter: Starting from a 4-bit counter studied in class, the design was expanded to 14 bits to support counting up to 9999. The module includes an up_down signal to determine the counting direction, an en signal to enable counting, and a load signal to initialize the counter with an external value (data_in). A conditional structure ensures that if data_in is greater than 9999, the counter resets



to 0. An additional output signal last is used to indicate when the counter reaches its maximum value. This module was written in Verilog using an always block sensitive to the rising edge of clk and rst.

```

module counter_9999 #(
    parameter WIDTH = 14,
    parameter LIMIT = 9999
)(
    input wire clk,
    input wire rst,
    input wire en,
    input wire load,
    input wire up_down, // 0 = up, 1 = down
    input wire [WIDTH-1:0] data_in,
    output wire last,
    output reg [WIDTH-1:0] q
);

always @(posedge clk or posedge rst) begin
    if (rst) begin
        q <= 0;
    end else if (load) begin
        if (data_in > LIMIT)
            q <= 0;
        else
            q <= data_in;
    end else if (en) begin
        if (up_down == 0) begin // count up
            if (q == LIMIT)
                q <= 0;
            else
                q <= q + 1;
        end else begin // count down
            if (q == 0)
                q <= LIMIT;
            else
                q <= q - 1;
        end
    end
end

assign last = (q == LIMIT);
endmodule

```

To visualize the counter output on 7-segment displays, a binary-to-BCD conversion module was implemented using the "double dabble" algorithm. This module outputs four 4-bit BCD digits representing units, tens, hundreds, and thousands. Each BCD digit is then passed to a decoder module that maps the value to the appropriate segments for a common-anode display. The segments are driven such that a '0' turns on a segment.

```

module bcd_to_seg (
    input wire [3:0] bcd,
    output reg [6:0] seg // seg[6] = a
);
always @(*) begin
    case (bcd)
        4'd0: seg = 7'b1000000;
        4'd1: seg = 7'b1111001;
        4'd2: seg = 7'b0100100;
        4'd3: seg = 7'b0110000;
        4'd4: seg = 7'b0011001;
        4'd5: seg = 7'b0010010;
        4'd6: seg = 7'b0000010;
        4'd7: seg = 7'b1111000;
        4'd8: seg = 7'b0000000;
        4'd9: seg = 7'b0010000;
        default: seg = 7'b1111111;
    endcase
end
endmodule

```

```

module binary_to_bcd (
    input wire [13:0] bin,           // hasta 9999 = 14 bits
    output reg [3:0] units, tens, hundreds, thousands
);
    integer i;
    reg [27:0] shift_reg;

    always @(*) begin
        shift_reg = 0;
        shift_reg[13:0] = bin;

        for (i = 0; i < 14; i = i + 1) begin
            if (shift_reg[17:14] >= 5)
                shift_reg[17:14] = shift_reg[17:14] + 3;
            if (shift_reg[21:18] >= 5)
                shift_reg[21:18] = shift_reg[21:18] + 3;
            if (shift_reg[25:22] >= 5)
                shift_reg[25:22] = shift_reg[25:22] + 3;
            if (shift_reg[27:26] >= 5)
                shift_reg[27:26] = shift_reg[27:26] + 3;

            shift_reg = shift_reg << 1;
        end

        thousands = shift_reg[27:24];
        hundreds  = shift_reg[23:20];
        tens      = shift_reg[19:16];
        units     = shift_reg[15:12];
    end
endmodule

```

Additionally, the system includes input from toggle switches SW[0–13] for data_in, with SW14 used to select the counting direction (up_down), SW15 to enable the counter, and SW16 to load data. The clock (clk) and reset (rst) signals are activated through push buttons KEY0 and KEY1 respectively. All modules

were instantiated in the top module, including the 9999 counter, the binary-to-BCD converter, and the 7-segment display drivers.

```

module counter_display_top (
    input wire clk, rst, en, load, up_down,
    input wire [13:0] data_in,
    output wire [6:0] HEX0, HEX1, HEX2, HEX3,
    output reg cicle
);

    wire [13:0] count;
    wire last;
    wire [3:0] u, d, c, m;

    // Instancia del contador
    counter_9999 #(.WIDTH(14), .LIMIT(9999)) U1 (
        .clk(clk),
        .rst(rst),
        .en(en),
        .load(load),
        .up_down(up_down),
        .data_in(data_in),
        .last(last),
        .q(count)
    );
endmodule

```

A simple sequential block was added to the top module to monitor the value of the counter. If the counter reaches 9999, the LEDG8 output is set to 1. When the counter reaches 8, LEDG8 is reset to 0. This logic is handled using a separate always block that reacts to the rising edge of the clock and reset signals.

```

counter_9999 #(.WIDTH(14), .LIMIT(9999)) U1 (
    .clk(clk),
    .rst(rst),
    .en(en),
    .load(load),
    .up_down(up_down),
    .data_in(data_in),
    .last(last),
    .q(count)
);

// Convertir a BCD para display
binary_to_bcd u2 (
    .bin(count),
    .units(u),
    .tens(d),
    .hundreds(c),
    .thousands(m)
);

// Módulos de display de 7 segmentos
bcd_to_7seg U3 (.bcd(u), .seg(HEX0));
bcd_to_7seg U4 (.bcd(d), .seg(HEX1));
bcd_to_7seg U5 (.bcd(c), .seg(HEX2));
bcd_to_7seg U6 (.bcd(m), .seg(HEX3));

// Encender LEDG8 cuando se llegue a 9999, apagar cuando llegue a 8
always @(posedge clk or posedge rst) begin
    if (rst)
        cicle <= 0;
    else begin
        if (count == 9999)
            cicle <= 1;
        else if (count == 8)
            cicle <= 0;
    end
end
end

```

Testbench

The testbench simulates a manual clock pulse (emulating button KEY0) using a task that toggles the clk signal. Several test cases were written: one to load a value close to 9999 and verify the wrap-around, one to test descending count, and one to validate LEDG8 (cicle) logic when reaching 9999 and 8. The \$monitor statement prints simulation data, and \$stop halts the simulation. (Insert waveform showing counting, loading, and cicle behavior)

```

module counter_9999_tb;

    parameter WIDTH = 14;
    reg clk, rst, en, load, up_down;
    reg [WIDTH-1:0] data_in;
    wire last;
    wire [WIDTH-1:0] q;

    // DUT
    counter_9999 #(WIDTH) DUT (
        .clk(clk),
        .rst(rst),
        .en(en),
        .load(load),
        .up_down(up_down),
        .data_in(data_in),
        .last(last),
        .q(q)
    );

    task pulse_clk;
    begin
        clk = 1; #5;
        clk = 0; #5;
    end
    endtask

    initial begin
        $monitor("t=%t | q=%d | cicle=%b",
            $time, q, (q == 9999 ? 1'b1 : (q == 8 ? 1'b0 : 1'bz)));

        // Inicialización
        clk = 0; rst = 1; en = 0; load = 0; up_down = 0; data_in = 0;
        #10;

        rst = 0;

        // Cargar 9998 y contar 1 (para llegar a 9999)
        data_in = 9998;
        load = 1;
        pulse_clk;
        load = 0;
        en = 1; up_down = 0;
        pulse_clk; // debe ir a 9999 - cicle = 1

        // Contar hacia abajo hasta 8
        up_down = 1;
        repeat (9991) pulse_clk; // 9999 - ... - 8

        pulse_clk; // q == 8 - cicle debe apagarse (0)

        $stop;
    end
endmodule

```

FPGA Implementation

Flow Status	Successful - Sun May 25 17:14:56 2025
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	counter
Top-level Entity Name	counter_display_top
Family	Cyclone IV E
Device	EP4CE115F29C7
Timing Models	Final
Total logic elements	184
Total registers	15
Total pins	48
Total virtual pins	0
Total memory bits	0
Embedded Multiplier 9-bit elements	0
Total PLLs	0

Conclusion/Issues

Cesar Eduardo Inda Cenicerros

Designing a parameterized counter in Verilog allows adapting the circuit to different counting ranges and facilitates reuse in future projects. Moreover, integrating it with displays and output elements improves visual interpretation of results on an FPGA.

Alonso Emmanuel López Macias.

Simulation through a testbench with controlled pulses allows accurate validation of the circuit's behavior under conditions similar to the physical environment, helping detect logic errors before hardware implementation.